

Spring RESTful Notes

1. `<mvc:annotation-driven />` is required.

2. `@RequestBody` is required even with `@RestController`.

3. `@ResponseBody` is not needed for `@RestController`

`@RestController = @Controller + @ResponseBody`
`@ResponseBody` is not needed with `@RestController`
However, `@RequestBody` is needed with `@RestController`. If omitted, there will be hidden mapping errors.

4. `@RestController` vs `@Controller`

<https://www.genuitec.com/spring-frameworkrestcontroller-vs-controller/>

Spring's annotation based MVC framework simplifies the process of creating RESTful web services. The key difference between a traditional Spring MVC controller and the RESTful web service controller is the way the HTTP response body is created. While the traditional MVC controller relies on the View technology, the RESTful web service controller simply returns the object and the object data is written directly to the HTTP response as JSON/XML.

Spring MVC REST Workflow

The following steps describe a typical Spring MVC REST workflow:

1. The client sends a request to a web service in URI form.
2. The request is intercepted by the DispatcherServlet which looks for Handler Mappings and its type.
 - The Handler Mappings section defined in the application context file tells DispatcherServlet which strategy to use to find controllers based on the incoming request.
 - Spring MVC supports three different types of mapping request URIs to controllers: annotation, name conventions and explicit mappings.
3. Requests are processed by the Controller and the response is returned to the DispatcherServlet which then dispatches to the view.

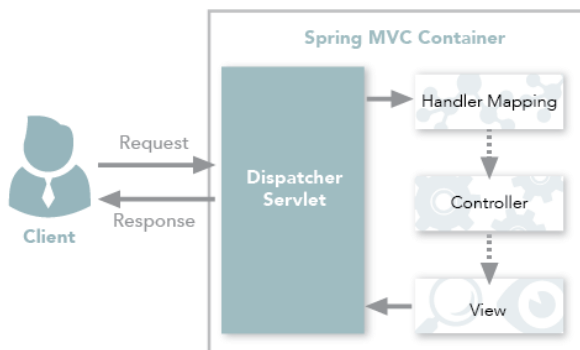


Figure 1: Spring MVC traditional workflow

In Figure 1, notice that in the traditional workflow the ModelAndView object is forwarded from the controller to the client. Spring lets you return data directly from the controller, without looking for a view, using the `@ResponseBody` annotation on a method. Beginning with Version 4.0, this process is simplified even further with the introduction of the `@RestController` annotation. Each approach is explained below.

Using the `@ResponseBody` Annotation

When you use the `@ResponseBody` annotation on a method, Spring converts the return value and writes it to the http response automatically. Each method in the Controller class must be annotated with `@ResponseBody`.

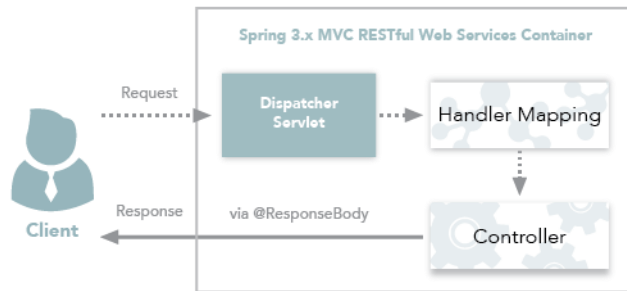


Figure 2: Spring 3.x MVC RESTful web services workflow

Behind the Scenes

Spring has a list of `HttpMessageConverters` registered in the background. The responsibility of the `HttpMessageConverter` is to convert the request body to a specific class and back to the response body again, depending on a predefined mime type. Every time an issued request hits `@ResponseBody`, Spring loops through all registered `HttpMessageConverters` seeking the first that fits the given mime type and class, and then uses it for the actual conversion.

- Add the `<context:component-scan>` and `<mvc:annotation-driven />` tags to the Spring configuration file.
- `<context:component-scan>` activates the annotations and scans the packages to find and register beans within the application context.
- **`<mvc:annotation-driven/>` adds support for reading and writing JSON/XML if the Jackson/JAXB libraries are on the classpath.**
- For JSON format, include the `jackson-databind` jar and for XML include the `jaxb-api-osgi` jar to the project classpath.